

CSE328: Portal Raycaster

1. Introduction

Raycasting is a rendering technique used to simulate a 3D perspective given a 2D map. This is done by casting rays from a viewer's viewpoint and extrapolating height, color, and shading information from the ray's first point of contact in the environment. This technique can frequently be found in early game engines, such as Wolfenstein 3D. This project extends the raycasting foundation with a portal system, which treats selected wall segments as windows into other regions of the space. Special collision rules have been introduced for these portal walls, teleporting the viewer to the linked portal's position and orientation.

2. Raycasting Engine

The raycasting engine operates on a per-column basis. For each vertical column of the screen, a ray is cast from the player's position in a direction determined by the player's facing angle and the column's horizontal offset from the screen center. The Digital Differential Analyzer (DDA) algorithm is then used to step the ray through the tile grid, finding the nearest wall intersection by advancing one tile boundary at a time (in either the X or Y direction).

Upon hitting a wall, the perpendicular distance between the player and the hit wall is computed. The height of the wall slice drawn for a given column is inversely proportional to the distance, such that closer walls produce taller slices and farther walls produce shorter slices. This creates the illusion of depth when all columns are rendered.

Walls are assigned a base color determined by their tile value (grey, red, blue, or green). Shading is then applied as a function of both distance and side axis (walls hit on their Y-facing side are darkened to give a sense of directionality). All walls fade toward a minimum brightness as distance increases, simulating the falloff of light across the environment.

3. Basic Portal System

Portals are defined as line segments placed within a cell's tile grid. This project adopts the cell and portal graph (CPG) model introduced by Lowe and Datta [1], in which a scene is partitioned into discrete cells connected by portals. Each portal stores its center position, facing angle, and a pointer to its linked partner portal. When two portals are linked, each stores the other's midpoint as an exit anchor, and a rotation angle is computed from the difference in their facing angles.

During raycasting, before each DDA step, the ray is tested against all portals in the current cell using a line segment intersection check. If the ray reaches a portal before hitting a wall, its position is remapped relative to the portal's partner's midpoint. Its direction is rotated by the precomputed rotation angle. Traversal then continues from this new position and direction within the destination cell, with the accumulated distance carried forward. [1]

Player crossing uses the same intersection logic. When the player's movement vector crosses a portal segment, the player's position and orientation are transformed, producing a seamless transition between cells.

4. Non-Adjacent & Same-Room Portals

Lowe and Datta [1] note that portals are not limited to connecting adjacent rooms, and any cell may be linked to any other (including to itself). This project implements both non-adjacent and adjacent portals, which work identically. Because each portal stores a direct pointer to its destination rather than relying on physical adjacency, the raycaster just traverses whatever cell the portal points to.

Same-room portals introduce a special case in which a ray or player may risk immediately re-crossing a portal when crossing on a given frame. This is handled by tracking the previously occupied cell and suppressing portal crossings that return within the same frame. As such, the player is not teleported back instantly and same-room traversal is stabilized.

5. Portal Rotation

When two portals are linked, a rotation angle is computed for each as the difference between their facing angles, plus π . This is to account for the fact that a ray exits through the front of the destination portal, which faces opposite to its inward portal. This rotation angle is used to both redirect the ray direction and rotate the player's orientation on crossing, so that the transition feels spatially consistent regardless of the angle between the portals.

After computing the rotation angle, the system checks whether the rotated tangent of the source portal aligns with the tangent of the destination. If the dot product is negative, the destination portal's endpoints are swapped, ensuring that the two segments wind in the same direction. Ablett et al. [2] similarly identify that the coordinate frame transformation between portal halves must account for orientation, and that the relationship between the local and destination sides is a key source of complexity in portal implementation.

6. Runtime Portal Editing

Portals can be repositioned and reoriented at runtime using the keyboard. Selecting a portal with Tab cycles through the portals in the current cell. The arrow keys translate the currently selected portal, and the comma and period keys (which occupy the same space on the keyboard as < and >) rotate the currently selected portal about its center. After any edit, the rotation angle between the two linked portals is recomputed, and the endpoint winding check is repeated. This keeps the link geometrically consistent as the portal moves.

7. Conclusion

This project successfully implemented a 2.5D portal raycasting engine in C, starting from a core DDA renderer, extending to adjacent and non-adjacent portals, and then runtime portal translation and rotation. The implementation follows the cell and portal graph paradigm of Lowe and Datta [1] and draws on the coordinate frame transformation concepts discussed by Ablett et al. [2], adapted to a raycasting context.

References

- [1] N. Lowe and A. Datta, "A technique for rendering complex portals," in *IEEE Transactions on Visualization and Computer Graphics*, vol. 11, no. 1, pp. 81–90, Jan.–Feb. 2005, doi: 10.1109/TVCG.2005.1.
- [2] D. Ablett, A. Cunningham, G. A. Lee and B. H. Thomas, "Portal Rendering and Creation Interactions in Virtual Reality," 2022 IEEE International Symposium on Mixed and Augmented Reality (ISMAR), Singapore, 2022, pp. 160–168, doi: 10.1109/ISMAR55827.2022.00030.